ways to do this is simply to write the code you would like to have. This allows you to design a clean, readable, self-documenting API.

For example, to find the next departure times from a given station, one very simple implementation might look something like this:

```
proposedTrainTimes = itineraryService.findNextDepartures(departure,
                                                         destination,
                                                         startTime);
```

Integrating this into the `@When` step, you'll get something like this:

```
List<LocalTime> proposedTrainTimes;

@When("I want to travel from $departure to $destination at $startTime")
public void whenIWantToTravel(String departure,
                              String destination,
                              LocalTime startTime) {
    ItineraryService itineraryService = new ItineraryService();
    proposedTrainTimes = itineraryService.findNextDepartures(departure,
                                                             destination,
                                                             startTime);
}
```

The When step

Passing in parameters from the When step

Finding the next departure times

Creating a new itinerary service

To get this acceptance criteria to pass, you'll implement the `findNextDepartures()` method. But before you do so, you need to change gears from acceptance testing to unit testing. As you'll see, acceptance testing is used to demonstrate the high-level, end-to-end behavior of an application, and unit testing is used to build up the components you'll use to implement this behavior. Acceptance tests typically use a full or near-full application stack, whereas unit tests concentrate on individual components

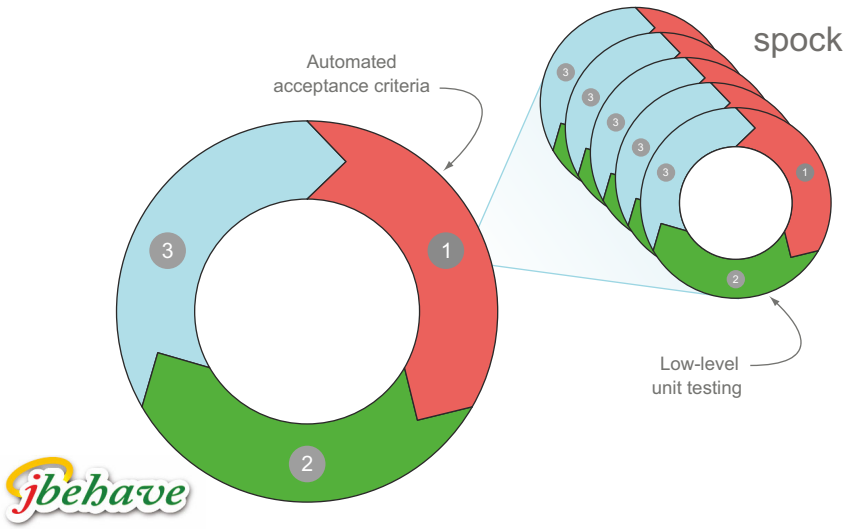


Figure 2.8 You'll typically need to write many low-level, TDD-style unit tests to get an automated acceptance criterion to pass. The example in this chapter uses JBehave for the acceptance criteria and Spock for the unit tests.